Fujisaki Okamoto on el gamal

Pushpendra Pal

Objective
Transform a CPA secure encryption scheme P.Σ into a CCA secure encryption scheme FO.Σ

1 Overview

The Fujisaki-Okamoto (FO) transform converts a CPA-secure public-key encryption scheme into a CCA-secure one by adding:

- Randomness extraction using hash functions
- Integrity verification to detect tampering
- Re-encryption check to ensure consistency

2 Example: Transforming El-gamal

Alice(Sender) | Bob(receiver)

- Choose x ϵ Z_q
- Choose h = g^x.

$\longleftarrow \overset{Publickey{:}h}{\longleftarrow\!\!\!\!\!\!\!\!\!\!\longleftarrow} \longleftarrow$

- Message m
- Choose rϵ Z_q
- $c_1 = g^r$
- $c_2 = m.h^r$

$\longrightarrow \overset{c_1,c_2}{\longrightarrow\!\!\!\!\!\!\!\!\!\!\longrightarrow} \longrightarrow$

- Receive (c_1, c_2)
- m = $\frac{c_2}{{c_1}^x}$
- Output: m

2.1 Problems with Malleability

ElGamal is multiplicatively malleable. Given a ciphertext $(c_1, c_2) = (g^r, \text{m} \cdot h^r)$ encrypting message m, an attacker can:

- Multiply by a constant: Create $(c_1, \text{k} \cdot c_2)$ which decrypts to $\text{k} \cdot \text{m}$
- Combine ciphertexts: Given two ciphertexts encrypting m_1 and m_2, create their "product" encrypting $m_1 \cdot m_2$
- Re-randomize: Add fresh randomness to create a different ciphertext for the same message

2.2 Concrete Attack Example

- Original ciphertext: $(g^r, \text{m} \cdot h^r)$ encrypting message m
- Attacker creates: $(g^r, 2 \cdot \text{m} \cdot h^r)$ by multiplying c_2 by 2
- This new ciphertext decrypts to 2m
- In a CCA setting, the attacker could query this modified ciphertext to a decryption oracle and learn 2m, then deduce m

This violates CCA security because the attacker can create valid ciphertexts related to the challenge ciphertext and use the decryption oracle to extract information.

2.3 Fujisaki Okamoto transform of El-gamal by Claude, (seems wrong)

Alice(Sender) | Bob(receiver)

- Choose x ϵ Z_q
- Compute h = g^x

$\longleftarrow$ $\xleftarrow{Publickey:h}$ $\longleftarrow$

$\longleftarrow$ $\xleftarrow{H_1,H_2}$ $\longleftarrow$

- **ENCRYPTION**
- Message: m
- Choose $\sigma\epsilon$ $\{0,1\}^n$
- r = $H_1(m||\sigma)$
- $c_1 = g^r$
- c_2 = (m $\oplus$ $H_2(\sigma||c_1)||\sigma)h^r$

$\longrightarrow$ $\xrightarrow{(c_1,c_2)}$ $\longrightarrow$

- **DECRYPTION**
- Receive (c_1,c_2)
- ($m'||\sigma$') = $\frac{c_2}{(c_1)^x}$
- Verify: c_1 ?= $g^{H_1(m'||\sigma')}$
- If NO: output **null**
- If YES: output m'$\oplus H_2(\sigma'||c_1)$
- READ THE SECOND POINT BELOW

2.3.1 Observations

- During decryption, m' = m $\oplus$ $H_1(\sigma||c_1)$, and σ' == σ, so then $c_1 \neq g^{H_1(m'||\sigma')}$, unless the Hash function is NOT second pre image resistant. Absurd!
- One thing that could explain this. After $\frac{c_2}{(c_1)^x}$, we get $m'||\sigma$', where σ' == σ and m' = m$\oplus H_2(\sigma'||c_1)$ and we already have c_1, so we can get $H_2(\sigma||c_1)$, hence we can decrypt c_2 here tiself, and get m. Then for verification of this step:// c_1 ?= $g^{H_1(m||\sigma)}$ //, that is

what is wrong above, using m' instead of m, but if we use m, then the verification succeeds.

  - Receive (c_1,c_2)
  - $(m'||\sigma) = \frac{c_2}{(c_1)^x}$
  - **m = m'**$\oplus H_2(\sigma||c_1)$, the same σ we got from previous step.
  - Verify: c_1 ?= $g^{H_1(m||\sigma)}$
  - If NO: output **null**
  - If YES: output **m**

- One more issue is the usage of **message m** inside H_1, which is not done in my version.

2.4 FJOK transform of El-gamal, my appraoch

- Choose $\alpha \epsilon Z_p$
- Compute h = g_α

Public key: h

H_1,H_2

- Message: m
- Choose x $\epsilon \{0,1\}^n$
- $c_1 = (g^r, [x||H_1(x)] \oplus h^r)$
- k = H_2(x)
- c_2 = m $\oplus$ k

(c_1,c_2)

- **DECRYPTION**
- Receive (c_1,c_2), let $(v_1, v_2) = c_1$
- $[x||H_1(x)] = \frac{v_2}{(v_1)^\alpha}$, extract x, call it x'.
- IF $H_1(x')$?= $H_1(x)$, then w.h.p. x=x', proceed
- k = $H_2(x)$
- m = $c_2 \oplus k$

3 Why This Achieves CCA Security (Everything from here is for CLAUDE version)

3.1 Detailed Malleability Problem in ElGamal

3.1.1 Why Standard ElGamal Fails Against CCA Attacks

The Multiplicative Structure Problem:
Standard ElGamal has the property that if (c_1, c_2) encrypts message m, then:

(c_1, k · c_2) encrypts message k · m for any constant k (c_1 · c_1', c_2 · c_2') encrypts m · m' if (c_1, c_2) encrypts m and (c_1', c_2') encrypts m'.

3.2 Concrete CCA Attack on Standard ElGamal:

Setup: Attacker wants to break message m* encrypted as (c_1*, c_2*) = (g^{r}*, m* · h^{r}*) Attack Steps:

- Choose a known value k ≠ 1 (e.g., k = 2)
- Create modified ciphertext: (c_1*, k · c_2*) = (g^{r}*, k · m* · h^{r}*)
- Submit this to the decryption oracle
- Oracle returns k · m* (since it's a valid ciphertext, just not the original)
- Compute m* = (k · m*) / k

3.2.1 Why This Works:

- The modified ciphertext (c_1*, k · c_2*) is perfectly valid ElGamal encryption
- It decrypts to k · m* using the standard ElGamal decryption algorithm
- The attacker learns m* without directly querying the challenge ciphertext
- This violates CCA security completely

3.2.2 Alternative Attack - Re-randomization:

- Pick random s and compute (c_1* · g^{s}, c_2* · h^{s})
- This encrypts the same message m* but with different randomness
- Query this "fresh-looking" ciphertext to the oracle
- Directly learn m*

3.3 How FO Transform Fixes the Malleability Problem

3.3.1 Key Security Properties:

- Randomness Binding: The randomness r is deterministically derived from both the message m and a random value σ using H_1(m —— σ). This prevents the attacker from controlling the randomness directly.

- Integrity Check: During decryption, we verify that $c_1 = g_1^H(\text{m'}——\sigma')$. This ensures the ciphertext hasn't been tampered with, as any modification would likely fail this check.

3.3.2 Attack Prevention in FO-ElGamal:

- Multiplication Attack: If attacker creates (c_1*, k · c_2*), the decryption gives some (m" $||\sigma$"), but the verification c_1* ?= g_1^H(m"$||\sigma$") fails because k · m* ≠ m* leads to different hash input
- Re-randomization Attack: Adding randomness (c_1* · g^s, c_2* · h^s) changes c_1*, but H_1(m* $||\sigma$*) still produces the original randomness, so c_1* · $g^s \neq g_1^H$(m*——σ*) and verification fails
- Any tampering is detected because the consistency check binds the ciphertext components together cryptographically.

3.3.3 Why the Re-encryption Check Works:

The check c_1 ?= g_1^H(m$||\sigma$) ensures that:

- The randomness used to create c_1 matches what would be derived from the decrypted message
- Any modification to either component breaks this relationship
- The hash function makes it computationally infeasible to find collisions that would pass the check

3.3.4 CCA Attack Resistance:

Scenario: Attacker has access to a decryption oracle and wants to decrypt a challenge ciphertext (c_1*, c_2*).
Why FO Transform Prevents This

- If the attacker queries a modified ciphertext (c_1', c_2') to the decryption oracle
- The re-encryption check c_1' ?= g_1^H(m'$||\sigma$') will likely fail for any meaningful modification
- The oracle returns **null**, giving the attacker no useful information
- For the original challenge ciphertext, the attacker can't query it directly (that would be cheating in the CCA game)

4 Why Hash Functions Are Crucial

The hash functions H_1 and H_2 are modeled as random oracles in the security proof:

- H_1 ensures the randomness is unpredictable but verifiable.
- H_2 provides additional randomness extraction.

Together, they bind the message, randomness, and ciphertext components This binding is what prevents malleability and enables the consistency check that makes CCA attacks fail.